HRMS-BACKEND-WEEKS- ONE-TWO-PHASE-ONE-TWO

1. Introduction

This project implements the backend of a Human Resource Management System (HRMS) using Node.js, Express, and MongoDB with Mongoose ODM.

The goal of Phase 1 – Week 2 is to build the core operational modules on top of the authentication foundation built in Week 1:

- Employee Management (Full CRUD Operations)
- Leave Request Management (Submit, Approve, Reject)
- Attendance Tracking (Check-In / Check-Out)
- Protected Routes with Role-Based Access Control

The system supports three user roles:

- Admin
- Manager
- Employee

Each role has different access permissions across all modules and endpoints.

2. System Architecture

The backend follows a layered architecture to separate responsibilities.

Typical structure:

```
hrms-project
│
├── app.js
├── .env
├── package.json
│
├── models
│   ├── Employee.js
│   ├── Leave.js
│   └── Attendance.js
│
├── controllers
│   ├── authController.js
│   ├── employeeController.js
│   ├── leaveController.js
│   └── attendanceController.js
│
├── routes
│   ├── authRoutes.js
│   ├── employeeRoutes.js
│   ├── leaveRoutes.js
│   └── attendanceRoutes.js
│
└── middleware
    └── authMiddleware.js
```

Explanation:

Layer	Role
app.js	Entry point – initializes server, connects database, registers routes
Routes	Define API endpoints and link them to controllers
Controllers	Contain business logic for each module
Models	Define MongoDB collection schemas using Mongoose
Middleware	Handle JWT authentication and role-based access
Database	MongoDB stores all system data

This architecture improves maintainability and scalability.

3. Technologies Used

Technology	Version	Purpose
Node.js	–	Backend JavaScript runtime
Express.js	^5.2.1	Web server and routing framework
MongoDB	–	NoSQL database for data storage
Mongoose	^9.2.4	ODM for schema definition and DB interaction
jsonwebtoken	^9.0.3	JWT generation and verification
bcryptjs	^3.0.3	Secure password hashing
dotenv	^17.3.1	Loading environment variables from .env file
cors	^2.8.6	Enabling cross-origin requests
nodemon	^3.1.14	Auto-restart server during development

4. Entry Point – app.js

The application starts from:

```
app.js
```

Responsibilities of this file:

- Load environment variables using dotenv
- Initialize Express application
- Configure express.json() middleware to parse JSON request bodies
- Connect to MongoDB using the dbConnection() async function
- Register all route modules under their base paths
- Expose a root health-check endpoint GET /

- Start the HTTP server on the configured port

Server initialization code:

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

const port = process.env.PORT || 5000;

async function dbConnection() {
  try {
    await mongoose.connect(process.env.DB_URL);
    console.log("MongoDB Connected!");
  } catch (error) {
    console.log(error);
  }
}
dbConnection();
```

Route registration:

```
const authRoutes       = require("./routes/authRoutes");
const employeeRoutes   = require("./routes/employeeRoutes");
const leaveRoutes      = require("./routes/leaveRoutes");
const attendanceRoutes = require("./routes/attendanceRoutes");

app.use("/api/auth",       authRoutes);
app.use("/api/employees",  employeeRoutes);
app.use("/api/leaves",     leaveRoutes);
app.use("/api/attendance", attendanceRoutes);
```

The server also exposes a health endpoint:

```
GET /
```

Response:

```
"HRMS Backend Running"
```

Server start:

```
app.listen(port, () => {
  console.log(`Server running at port ${port}`);
});
```

5. Environment Variables – .env

The .env file stores sensitive configuration values that should never be hardcoded.

```
PORT=5000
DB_URL=mongodb://localhost:27017/hrms
JWT_SECRET=hrmssecret
```

Variable	Value	Purpose
PORT	5000	Port the server listens on

DB_URL	mongodb://localhost:27017/hrms	MongoDB connection string
JWT_SECRET	hrmssecret	Secret key for signing JWT tokens

These values are loaded at startup using dotenv and accessed throughout the application via process.env.

6. Database Models

The system defines three main Mongoose models that map to MongoDB collections.

6.1 Employee Model

Represents employees in the organization.

Fields:

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
name	String	Employee full name
email	String	Unique email address
password	String	bcrypt-hashed password (never plain text)
role	Enum	admin / manager / employee (default: employee)

Example role definition:

```
role: {
  type: String,
  enum: ['admin', 'manager', 'employee'],
  default: 'employee'
}
```

Passwords are never stored in plain text. They are hashed using bcryptjs before being saved to the database.

6.2 Leave Model

Represents leave requests submitted by employees.

Fields:

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
employee_id	ObjectId	Reference to the Employee collection
start_date	Date	Start date of the leave

end_date	Date	End date of the leave
status	Enum	pending / approved / rejected (default: pending)

6.3 Attendance Model

Represents daily employee attendance records.

Fields:

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
employee_id	ObjectId	Reference to the Employee collection
date	Date	Date of the attendance record
check_in	Date	Timestamp of employee check-in
check_out	Date	Timestamp of employee check-out

7. Model Relationships

Relationships are defined using ObjectId references in MongoDB.

Example:

```
employee_id: {
  type: mongoose.Schema.Types.ObjectId,
  ref: 'Employee',
  required: true
}
```

Meaning:

- One employee can have many attendance records
- Each attendance record belongs to one employee
- One employee can submit many leave requests
- Each leave request belongs to one employee

8. Input Validation

Before inserting data into the database, the system validates user input.

Validation ensures:

- Email format is correct
- Password length is sufficient (minimum 6 characters)
- Required fields (name, email, password) are provided
- Dates are valid for leave requests

Example:

```
if (password.length < 6) {
  errors.push("Password must be at least 6 characters");
}
```

This prevents invalid data from entering the system.

9. Authentication System

The system uses JWT (JSON Web Token) for stateless authentication.

Workflow:

- User sends POST /api/auth/login with email and password
- Server looks up the employee by email in MongoDB
- Server compares submitted password with bcrypt hash
- On success, server generates a signed JWT token
- Client stores the token and sends it with future requests
- Protected routes verify the token before granting access

Example token generation:

```
const token = jwt.sign(
  { id: employee._id, email: employee.email, role: employee.role },
  process.env.JWT_SECRET,
  { expiresIn: '7d' }
);
```

Token payload contains:

- Employee _id
- Email
- Role (admin / manager / employee)

10. Password Security

Passwords are protected using bcryptjs hashing with a salt round of 10.

During registration:

```
const hashed = await bcrypt.hash(password, 10);
```

During login:

```
const match = await bcrypt.compare(password, employee.password);
```

This ensures passwords cannot be reversed or stolen even if the database is compromised.

11. Middleware System

Middleware functions run before reaching the controller and handle cross-cutting concerns.

11.1 Authentication Middleware

File: middleware/authMiddleware.js

Responsibilities:

- Read JWT token from the Authorization header (Bearer TOKEN)
- Verify the token using process.env.JWT_SECRET
- Decode the payload and attach user info to req.user
- Reject any request with an invalid or missing token (401 Unauthorized)

Example:

```
const token = req.headers.authorization?.split(" ")[1];
if (!token) return res.status(401).json({ message: 'No token provided' });

const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = decoded;
```

11.2 Role-Based Access Control

Role middleware restricts specific endpoints to specific roles.

Example – Admin Only:

```
const allowAdmin = (req, res, next) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ message: 'Access denied' });
  }
  next();
};
```

Access control summary:

Role	Can Access
Admin	All endpoints (employees, leaves, attendance, auth)
Manager	View employees, approve/reject leaves, view attendance
Employee	Submit leave requests, record own attendance, view own data

12. API Routes

All routes are registered in app.js and protected by middleware where required.

12.1 Auth Routes – /api/auth

Method	Endpoint	Description	Access
POST	/api/auth/register	Register a new employee	Public

POST	/api/auth/login	Login and receive JWT token	Public

12.2 Employee Routes – /api/employees

Method	Endpoint	Description	Access
GET	/api/employees	Get all employees	Admin / Manager
POST	/api/employees	Add new employee	Admin
PUT	/api/employees/:id	Update employee data	Admin
DELETE	/api/employees/:id	Delete employee	Admin

12.3 Leave Routes – /api/leaves

Method	Endpoint	Description	Access
POST	/api/leaves	Submit a leave request	Employee
GET	/api/leaves	View all leave requests	Admin / Manager
PUT	/api/leaves/:id/approve	Approve a leave request	Admin / Manager
PUT	/api/leaves/:id/reject	Reject a leave request	Admin / Manager

12.4 Attendance Routes – /api/attendance

Method	Endpoint	Description	Access
POST	/api/attendance/checkin	Record employee check-in	Employee
POST	/api/attendance/checkout	Record employee check-out	Employee
GET	/api/attendance	View all attendance records	Admin / Manager

13. Controllers (Business Logic)

Each controller file contains the logic for one module.

13.1 authController.js

- register: Validates input, hashes password with bcryptjs, creates new Employee document in MongoDB, returns success message
- login: Finds employee by email, compares password using bcrypt.compare, generates and returns a signed JWT token

13.2 employeeController.js

- getEmployees: Queries MongoDB for all employee documents and returns the list

- addEmployee: Validates and creates a new employee record with a hashed password
- updateEmployee: Finds employee by ID and updates the provided fields
- deleteEmployee: Removes an employee document by its ID

13.3 leaveController.js

- createLeave: Creates a new leave request with status set to 'pending' and links it to the logged-in employee
- getLeaves: Returns all leave request documents from MongoDB
- approveLeave: Finds a leave request by ID and sets its status to 'approved'
- rejectLeave: Finds a leave request by ID and sets its status to 'rejected'

13.4 attendanceController.js

- checkIn: Creates a new attendance record with the current date and check-in timestamp for the logged-in employee
- checkOut: Finds today's attendance record for the employee and updates the check-out timestamp
- getAttendance: Returns all attendance records from MongoDB

14. Error Handling

The system uses try/catch blocks in each controller to handle errors consistently.

All error responses follow a standardized JSON format:

```
{
  "success": false,
  "message": "Error description"
}
```

Example controller error handling:

```
try {
  // business logic
} catch (error) {
  res.status(500).json({ success: false, message: error.message });
}
```

This improves debugging and ensures the API always returns a readable response.

15. Request Flow

Example request: GET /api/employees (Admin Only)

Flow:

```
Client Request
 |
 v
Express Server (app.js)
```

```
 |
 v
Route  (/api/employees)
 |
 v
Auth Middleware  →  verifies JWT token
 |
 v
Role Middleware  →  checks Admin / Manager role
 |
 v
Controller  (employeeController.getEmployees)
 |
 v
MongoDB  (Employee.find())
 |
 v
JSON Response returned to Client
```

If authentication fails at any middleware step, the request is rejected before reaching the controller.

16. Security Features

The system implements multiple layers of security:

Security Feature	Implementation
Password Hashing	bcryptjs with salt rounds of 10
JWT Authentication	Signed tokens with expiry, verified on every request
Role-Based Access	Middleware enforces allowed roles per endpoint
Input Validation	Manual checks before any DB write operation
Environment Variables	Sensitive config (DB_URL, JWT_SECRET) stored in .env
Error Handling	Standardized JSON error responses via try/catch

These practices ensure the system is secure and production-ready.

17. Conclusion

Week 2 of Phase 1 of the HRMS backend successfully implements:

- Full Employee CRUD Management with role-based protection
- Leave Request System with submit, approve, and reject workflow
- Attendance Tracking with check-in and check-out functionality
- JWT-secured API with role-based access on all protected endpoints
- MongoDB database integration with Mongoose schemas and references

The architecture is designed to be scalable, allowing future phases to add:

- Monthly attendance and leave reports
- Employee self-service dashboard
- Automated leave balance calculation
- Notification system for leave approvals and rejections